

Lab 3

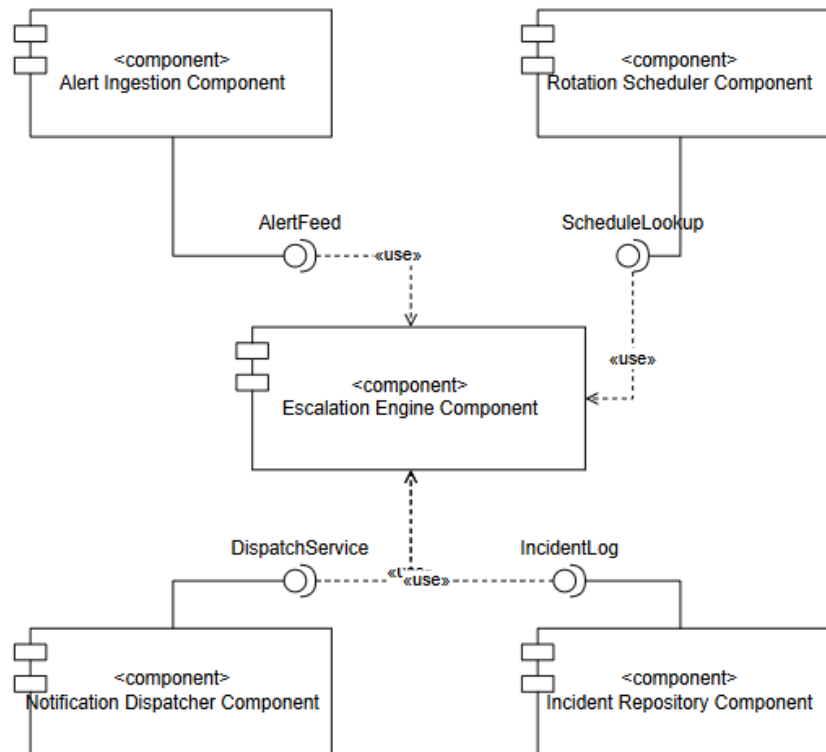
Name: Krithika.S

SRN: PES2UG24CS238

Incident Escalation & On-Call Rotation Engine

UML Component Diagram

Incident Escalation & On-Call Rotation Engine



Lab 3: Architectural Pattern Justification

Incident Escalation & On-Call Rotation Engine

Architecture Selection: I chose Microservices Architecture for the Incident Escalation & On-Call Rotation Engine system.

Architectural Choice

I used a Microservices Architecture where the Alert Ingestion, Rotation Scheduler, Escalation Engine, Notification Dispatcher, and Incident Repository work as separate services. They communicate through well-defined interfaces.

Reasons for the Choice

- 1. Independent Scaling:** Different parts of the system can receive different amounts of load. During a large number of incidents, the Notification Dispatcher may need more resources than the Rotation Scheduler. With microservices, I can scale only the service that needs it.
- 2. Fault Isolation and Independent Deployment:** If one service has a problem, it should not stop the complete alerting and escalation process. Separating the services helps isolate failures. I can also update the Escalation Engine without changing the other services.

Security Advantage

Each service can have its own authentication and authorization. For example, the Rotation Scheduler can accept requests only from the verified Escalation Engine. This helps reduce the impact if one service is compromised.

Performance Benefit

NFR-001 requires alert dispatch through webhook, SMS, and email to start within 3 seconds of alert ingestion. Since the Notification Dispatcher is a separate service, it can handle the notification channels in parallel and can be scaled when the alert load increases. This helps the system meet the required response time.